Краткая теория:

- Что такое:
  - виртуальная память
  - страницы (pages)
  - page fault
  - swap
- Почему процессу "кажется", что у него своя память

📌 Мини-вопросы:

- Почему программа может использовать больше RAM, чем есть физически?
- Что происходит при нехватке памяти?

🛠️ **Задание 1: Смотрим память системы**

Выполнить:

```
free -h
```

Разобрать:

- total / used / free
- buff/cache
- available

📌 Вопросы:

- Чем отличается free от available?
- Почему used ≠ реально занятая память?

🛠️ **Задание 2: Анализ процессов**

Top или htop

Найти:

- процесс с максимальным потреблением памяти
- значения:
  - VIRT
  - RES
  - SHR

📌 Вопросы:

- Почему VIRT >> RES?
- Что реально занимает RAM?

🛠️ **Задание 3: Изучаем конкретный процесс**

1. Запустить программу:

```
sleep 300
```

2. Найти PID:

```
ps aux | grep sleep
```

3. Изучить:

```
cat /proc/<PID>/status
```

Обратить внимание на:

- VmSize
- VmRSS
- VmData
- VmSwap

🛠️ **Задание 4: Карта памяти процесса**

cat /proc/<PID>/maps

📌 Разобрать:

- heap
- stack
- shared libraries

🛠️ **Задание 5: Python-скрипт (нагрузка памяти)**

Создать файл:

```
# memory_test.py
arr = []

while True:
    arr.append('A' * 10**6)  # ~1MB
```

Запустить:

```
python3 memory_test.py
```

В другом терминале:

```
top
```

📌 Наблюдать:

- рост RES
- поведение системы

📌 Вопросы:

- Что происходит при росте памяти?
- Когда процесс "умирает"?

🛠️ **Задание 6 (теоретико-практическое):**

Обсудить:

- что происходит при:
  - нехватке RAM
  - активном swap
- почему без sudo нельзя управлять swap, но можно его наблюдать

📌 К концу занятий:

- объяснить разницу:
  - VIRT / RES
  - RAM / swap
- анализировать память процесса через /proc
- понимать, как ОС "обманывает" процесс (виртуальная память)